Model-based Adaptation to Expanding Action Spaces in Reinforcement Learning

Duong Son Thong
Deakin University
Australia
Email: s223593948@deakin.edu.au

Dr Thommen George Karimpanal
Deakin University
Australia
Email: thommen.karimpanalgeorge@deakin.edu.au

***Abstract*—Reinforcement Learning (RL) agents are typically trained under the assumption of a fixed action space, which limits their adaptability when new actions are introduced. Retraining from scratch is costly and inefficient, motivating the need for methods that can reuse prior knowledge. In this work, we propose a model-based adaptation framework for expanding action spaces that builds on the Q-learning algorithm. Unlike prior approaches that rely on auxiliary demonstrations or continuous embeddings of actions, our method is based on learning a partial transition model and directly reusing previously learned value functions. We propose three extensions: (i) a partial model that learns and predicts transitions of new actions, (ii) encouraging the agent to select new actions , and (iii) biasing action selection only if it improves the state value. Experiments in GridWorld using both tabular Q-learning and DQN show faster adaptation, higher discounted returns, and improved safety compared to retraining from scratch.**

I. Introduction

Reinforcement Learning (RL) [1] has achieved remarkable progress in sequential decision-making problems such as robotics, dialogue systems, and recommender platforms [2]. However, most RL algorithms assume a fixed action space during training. In real-world settings, this assumption often fails. Robots may gain new tools, software systems may introduce additional commands, and agents may need to adapt to evolving environments.

When the action space expands, retraining from scratch is computationally expensive and sample inefficient. This raises the question of how an agent can efficiently adapt to newly introduced actions while reusing previously acquired knowledge.

Existing approaches in lifelong and continual reinforcement learning often rely on action embeddings, auxiliary demonstrations, or human guidance [3]–[5]. These methods can be effective in large or structured action spaces where newly introduced actions share similarities with existing ones. However, their performance can degrade when new actions behave as outliers with respect to the previously learned action set. In contrast, we propose a lightweight model-based framework that reuses previously learned value functions and learns a partial transition model only for newly introduced actions.

Our main contributions are:

- A partial-model-based framework for adapting to newly introduced actions without retraining from scratch.
- A novel action filtering condition that integrates new actions only when they improve predicted state value.
- Empirical validation in GridWorld environments demonstrating faster adaptation, higher returns, and improved safety.

II. Literature review

A. Model-Based Reinforcement Learning and Adaptation

The utility of parametric models in Reinforcement Learning (RL) has been a subject of significant debate, particularly regarding their efficiency compared to experience replay. Hasselt et al. [6] argue that while model-based approaches share the ability to plan, using computation without additional data to improve behavior, they often face stability issues. Specifically, when a model is used for credit assignment (updating value functions or policies), modeling errors can lead to ”catastrophic learning updates” because fictional transitions may corrupt the value estimates of real states. They hypothesize that under many conditions, simply replaying real experience is superior to planning with an inaccurate model for credit assignment.

However, a key distinction arises when models are used for immediate behavior rather than credit assignment. Hasselt et al. [6] demonstrate that ”planning in the now”, using a model to inform action selection at the current state, is highly beneficial and less likely to be harmful, as the resulting plan is not blindly trusted as real experience for long-term policy updates. This is particularly relevant in ”rich domains” where the current state may not have been observed before, making standard replay insufficient for determining behavior.

Our work builds on this insight by applying model-based planning to the specific challenge of expanding action spaces. Unlike prior methods that rely on continuous action embeddings or human-in-the-loop supervision , we propose a partial model framework. This approach mitigates the ”Deadly Triad” risks of divergence discussed by Hasselt et al. [6] by strictly limiting the model’s scope to newly introduced actions.

By implementing an action filtering condition where a new action is only executed if the partial model predicts a transition to a higher-value state we effectively ”plan forward for behavior” without the risks of ”planning forward for credit assignment”. This ensures that our agent maintains the stability of its previously learned value functions while safely

integrating new capabilities, a practical necessity in evolving real-world environments.

B. Model-Based Reinforcement Learning with Structured Action Spaces

Recent work has explored how model-based reinforcement learning can be extended to more structured action spaces beyond purely discrete or continuous settings. Zhang et al. [7] proposed Dynamics Learning and Predictive Control with Parameterized Actions (DLPA), the first model-based reinforcement learning framework for Parameterized Action Markov Decision Processes (PAMDPs), where each discrete action is associated with a continuous parameter. Their method learns a parameterized-action-conditioned dynamics model and performs planning via a modified Model Predictive Path Integral (MPPI) control algorithm. By explicitly modeling the coupling between discrete actions and continuous parameters, DLPA achieves strong sample efficiency and asymptotic performance across a range of PAMDP benchmarks.

Importantly, DLPA demonstrates that model-based planning can be effective in complex action spaces when the structure of the action space is known a priori. However, the setting considered in DLPA differs fundamentally from the problem addressed in this work. DLPA assumes a fixed set of discrete actions, each with predefined parameter spaces, and relies on learning a full environment model to support long-horizon planning. In contrast, our setting focuses on external action expansion, where new discrete actions are introduced after initial training and no prior structural relationship between old and new actions is assumed. Learning a full model over the expanded action space in such scenarios can be costly and potentially unstable.

Furthermore, DLPA uses learned models both for planning and implicit credit assignment through long-horizon rollouts, whereas our approach explicitly avoids propagating model errors into value updates. By restricting the model's role to predicting the immediate outcomes of newly introduced actions and integrating them only when they are predicted to improve the existing value function, our method follows a more conservative and value-preserving strategy. As such, DLPA highlights the power of model-based planning in structured action spaces, while our work complements this line of research by addressing safe and efficient adaptation in dynamically expanding discrete action spaces, where partial modeling and value reuse are essential.

C. Lifelong RL.

In lifelong reinforcement learning [3], an agent is required to solve a sequence of tasks where the action set can dynamically change over time. The central challenge is how to efficiently reuse knowledge from past experience to accelerate adaptation to newly introduced actions. Chandak [3] proposed LAICA within the lifelong Markov decision process (L-MDP) framework, which separates adaptation and improvement phases to incorporate new actions. Unlike traditional lifelong RL, where both state and action sets are fixed, their formulation explicitly accounts for expanding action sets.A key insight of their work is to leverage an underlying latent structure in the action space, which allows the policy to generalize to novel actions by reasoning about similarities with existing ones. By parameterizing the policy in this structure-invariant way, their approach avoids reinitializing parameters whenever new actions are introduced, reducing sample inefficiency. However, their approach relies on learning continuous embeddings of actions, while our research focuses on reusing previously learned policies and value functions directly.

D. Zero-Shot Generalization.

Another work studies zero-shot generalization to unseen actions [4]. In this setting, an agent trained on one set of actions must adapt to new actions without retraining. Jain et al [3] introduced hierarchical variational autoencoders (HVAE) to learn representations of actions from side observations such as videos or trajectories, enabling policies to generalize to new action choices. While promising, this approach assumes access to auxiliary information about unseen actions beforehand, which may not always be available in real-world applications. In addition, their experiments on environments like CREATE, Shape Stacking, and Recommender Systems showed that HVAE-based policies outperform non-hierarchical baselines and nearest-neighbor methods, highlighting the benefit of structured action embeddings. Regularization strategies such as action subsampling and entropy maximization were also critical to prevent overfitting to training actions, thereby improving generalization to new ones. However, they found that generalization becomes weaker when the test actions are very different from the training actions, showing a limit in distribution. Importantly, training policies again from the beginning was shown to be much more costly than using zero-shot generalization, which makes zero-shot more useful in domains where action sets change often. In contrast, our work aims to adapt efficiently without requiring such prior demonstrations, relying instead on policy/value reuse.

Taken together, lifelong RL and zero-shot generalization represent complementary strategies for handling expanding action spaces. Lifelong RL emphasizes incremental adaptation as new actions arrive, often with theoretical guarantees under the L-MDP formulation, whereas zero-shot approaches attempt immediate transfer without additional interaction. Bridging these two perspectives is essential for building agents that can both reuse prior value knowledge and exploit structured representations to cope with entirely novel action choices.

E. Human-in-the-Loop Action Expansion.

Mandel et al. [5] addressed the problem of where to add actions in human-in-the-loop reinforcement learning. They proposed the Expected Local Improvement (ELI) heuristic to determine high-impact states where human effort should be focused when introducing new actions. This approach minimizes wasted human input but requires continuous external guidance and assumes that humans generate the new actions. By contrast, our problem formulation assumes new actions

emerge automatically, e.g., due to environmental updates, and investigates how agents can autonomously leverage past knowledge to adapt. Their work also showed that without careful guidance, humans may spend effort adding actions in states that have little effect on long-term performance, reducing overall efficiency. ELI addresses this by selecting states that are most likely to give immediate improvements, which helps learning in very large action spaces. However, the method still depends on expert availability and can struggle when human-provided actions are poor or inconsistent. Later studies confirm that this framework improves results in simulated domains, but it remains less practical in scenarios where actions evolve without direct human design.

F. Dual-Adaptive.

In incremental reinforcement learning [8], the state and action spaces can continually expand as the environment evolves, making it impractical to assume a fixed environment. The main challenge is to efficiently explore newly introduced transitions while retaining knowledge from previous training. To address this, Ding et al. [8] proposed the Dual-Adaptive ϵ-Greedy Exploration (DAE) framework, which employs a Meta Policy to dynamically adjust the exploration rate and an Explorer to prioritize least-tried actions. Unlike traditional ϵ-greedy exploration that samples actions randomly, DAE adaptively balances exploitation and exploration, enabling efficient adaptation to expanding environments. Their experiments on synthetic benchmarks and Atari games demonstrated that DAE achieved over 80% performance improvement compared to eight baselines. While their method focuses on exploration efficiency in incremental environments, our research emphasizes reusing previously learned policies and value functions for safe and efficient adaptation when new actions are introduced.

G. Continual RL.

The broader field of continual reinforcement learning [9] provides a unifying framework for adaptation across non-stationary environments. CRL methods are commonly categorized into policy-focused, experience-focused, dynamic-focused, and reward-focused approaches, each balancing the trade-offs among stability, plasticity, and scalability. Our research is most closely aligned with policy-focused methods, especially policy reuse and decomposition, since the primary challenge is to accelerate learning in the presence of an expanded action set while avoiding costly retraining.

In contrast to these existing approaches, our method takes a different perspective. Instead of relying on continuous action embeddings [4], auxiliary demonstrations [4], or continuous human guidance [5], we design a framework that builds only a *partial model* for the newly introduced action while reusing the previously learned value functions and policies for the original action set. This makes our approach both simpler and more sample efficient than full-model learning or embedding-based methods, since it avoids relearning the entire action space. Moreover, by introducing an explicit *action filtering condition* (Eq. 7), we ensure that a new action is integrated only if it demonstrably improves the predicted state value. This leads to faster adaptation, reduced harmful exploration, and improved safety. Overall, our framework directly addresses the challenge of external action expansion by combining the efficiency of knowledge reuse with the stability of value-preserving action selection, offering a practical and lightweight alternative to prior continual RL methods

III. Methodology

Our approach builds on tabular Q-learning and DQN and introduces mechanisms to integrate new actions without discarding prior knowledge. Instead of learning a full environment model, we learn a partial model that predicts transitions for newly introduced actions only.

A. Q-learning Background

The Q-learning update rule is defined as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right], \tag{1}$$

where α is the learning rate and γ is the discount factor.

B. Deep Q-Network (DQN)

While tabular Q-learning is effective for small state spaces, it becomes infeasible in high-dimensional or continuous environments. Deep Q-Networks (DQN) address this limitation by approximating the action-value function $Q(s, a)$ with a neural network parameterized by θ.

Given a transition (s_t, a_t, r_t, s_{t+1}), DQN updates the network by minimizing the temporal-difference error:

$$y_t^{\text{DQN}} = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a'), \tag{2}$$

where θ^- denotes the parameters of a target network that is periodically updated from the online network.

The loss function is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(y_t^{\text{DQN}} - Q_\theta(s_t, a_t) \right)^2 \right]. \tag{3}$$

By using experience replay and a target network, DQN stabilizes training and enables Q-learning to scale to complex environments.

C. Multi-Layer Perceptron (MLP)

To model environment dynamics and support decision making beyond direct interaction, we employ a Multi-Layer Perceptron (MLP) [10] as a function approximator. An MLP consists of multiple fully connected layers that apply nonlinear transformations to the input, enabling it to capture complex relationships between states and actions.

In this work, the MLP is used to approximate the transition function of newly introduced actions. Given a state s and an action a, the MLP predicts the corresponding next state $\hat{s}'$:

$$\hat{s}' = f_\phi(s, a), \tag{4}$$

where f_ϕ denotes the MLP parameterized by ϕ.

By learning this transition model from experience, the MLP provides a predictive signal that is used to guide action selection. This allows the agent to assess the potential impact of new actions before executing them in the environment, thereby reducing unnecessary exploration and improving adaptation efficiency.

D. Encouraging Exploration for New Actions in Tabular Setting

We reuse the learned Q-values for old actions:

$$Q_{\text{new}}(s, a) \leftarrow Q_{\text{old}}(s, a), \quad \forall a \in \mathcal{A}. \tag{5}$$

New actions are initialized optimistically:

$$Q_{\text{new}}(s, a_{\text{new}}) = \max_{a \in \mathcal{A}} Q_{\text{old}}(s, a). \tag{6}$$

E. Encouraging Exploration for New Actions in DQN Setting

In the deep setting, we reuse the learned parameters of the Q-network for all previously available actions:

$$\theta_{\text{new}} \leftarrow \theta_{\text{old}}. \tag{7}$$

To encourage exploration of newly introduced actions, we bias the action selection toward these actions under an ϵ-greedy policy:

$$\pi(a \mid s) = \begin{cases} \text{Uniform}(\mathcal{A}_{\text{new}} \cup \mathcal{A}^*(s)), & \text{with probability } \epsilon, \\ \text{Uniform}(\mathcal{A}^*(s)), & \text{otherwise,} \end{cases} \tag{8}$$

where

$$\mathcal{A}^*(s) = \arg\max_{a \in \mathcal{A}_{\text{old}}} Q_\theta(s, a). \tag{9}$$

F. Partial Model and Action Filtering

A partial model predicts the next state s'_{model} for new actions:

$$s'_{\text{model}} \leftarrow \text{MLP}(s, a_{\text{new}}). \tag{10}$$

A new action is executed only if:

$$V_{\text{old}}(s'_{\text{model}}) > V_{\text{old}}(s). \tag{11}$$

G. Algorithms

Algorithm 1 Optimistic Initialization for New Actions in Tabular Setting

Require: Base Q-table Q_{old}, total actions $|\mathcal{A}|$

Copy old values: $Q(s, a) \leftarrow Q_{\text{old}}(s, a)$ for all $a \in \mathcal{A}_{\text{old}}$ ▷ Q_{old}: the Q-values learned before new actions are added

Compute $V_{\text{old}}(s) \leftarrow \max_{a \in \mathcal{A}_{\text{old}}} Q_{\text{old}}(s, a)$ ▷ $V_{\text{old}}(s)$: best value achievable in s using only old actions

for each new action $a_{\text{new}} \in \mathcal{A} \setminus \mathcal{A}_{\text{old}}$ **do**

    $Q(s, a_{\text{new}}) \leftarrow V_{\text{old}}(s)$ ▷ a_{new}: newly introduced action, initialized optimistically with $V_{\text{old}}(s)$

end for

Algorithm 2 Biased ϵ-Greedy Action Selection for DQN

Require: State s, exploration rate ϵ, Q-network Q_θ

Require: Old action set $\mathcal{A}_{\text{old}}$, expanded action set $\mathcal{A}_{\text{new}}$

Compute Q-values for all actions:

$$Q(s, a) \leftarrow Q_\theta(s, a), \quad \forall a \in \mathcal{A}_{\text{new}}$$

Compute greedy action set:

$$\mathcal{A}^*(s) \leftarrow \arg\max_{a \in \mathcal{A}_{\text{new}}} Q(s, a)$$

if $\text{Uniform}(0, 1) < \epsilon$ **then**

    Sample $a_{\text{new}} \sim \text{Uniform}(\mathcal{A}_{\text{new}} \setminus \mathcal{A}_{\text{old}})$

    return $\text{Uniform}(\{a_{\text{new}}\} \cup \mathcal{A}(s))$

else

    return $\text{Uniform}(\mathcal{A}^*(s))$

end if

Algorithm 3 Training with MLP transition model (DQN Setting)

Require: Q-network Q_θ, target network Q_{θ^-}, replay buffer $\mathcal{D}$

Require: Transition model (MLP) f_ϕ, exploration schedule ϵ

Require: Maximum episodes E, maximum steps T

for episode $ep = 1, 2, \ldots, E$ **do**

    Initialize state $s \leftarrow s_{\text{start}}$

    for step $t = 1, 2, \ldots, T$ **do**

        Sample action a using ϵ-greedy policy with new-action encouragement

        if $a \in \mathcal{A}_{\text{new}}$ **and** transition model is available **then**

            Predict next state $\hat{s}' \leftarrow f_\phi(s, a)$

            if $\max_{a'} Q_\theta(\hat{s}', a') \leq \max_{a'} Q_\theta(s, a')$ **then**

                Reject a, fallback to old-action policy

                $a \leftarrow \epsilon\text{-greedy}(Q_\theta(s, \cdot))$

            else

                Accept a

            end if

        end if

        Execute action a: observe s', reward r, termination flag $done$

        if $a \in \mathcal{A}_{\text{new}}$ **then**

            Store transition (s, a, s') in transition model buffer

        end if

        Store transition $(s, a, r, s', done)$ in replay buffer $\mathcal{D}$

        Update Q-network using a minibatch from $\mathcal{D}$

        Soft-update target network parameters

        $s \leftarrow s'$

        if $done$ **then**

            break

        end if

    end for

    Decay exploration rate ϵ

end for

IV. Experiments

A. Implementation Details

We evaluate our proposed method in a GridWorld environment designed to simulate the introduction of a new action after initial training. The agent is first trained with a fixed action set until convergence, after which an additional action is introduced. To ensure comparability, we implement both our method and baseline Q-learning with identical hyperparameters. Specifically, we adopt the standard Q-learning update with a learning rate $\alpha = 0.1$, discount factor $\gamma = 0.95$, and ϵ-greedy exploration with $\epsilon = 0.1$. For our method, new actions are initialized optimistically according to Equation (13), and the new action selection strategy described in Algorithm 1(in tabular setting) or Algorithm 2 (in DQN setting) is applied. Each experiment is repeated for 5 independent runs with different random seeds, and we report the mean performance with standard deviation.

B. Baselines

We compare against:

- Q-learning with original actions
- Retraining from scratch with expanded actions
- Naive expansion without knowledge reuse

C. Metrics

Performance is evaluated using discounted return and total number of collisions (bumps).

V. Results and Comparison

We compare our method against four baselines: (i) Q-learning with the original 4 actions, (ii) Q-learning retrained from scratch with 5 actions after the new action is introduced, (iii) continuing Q-learning with the expanded 5-action set but without knowledge reuse, and (iv) adapting to 5 actions by only reusing the old Q-table without partial model learning. Performance is evaluated using two key metrics:

- **Discounted Return**: measures cumulative efficiency and adaptation speed.
- **Total Bumps**: counts the number of times the agent collides with obstacles, serving as a proxy for safety during learning.

Small GridWorld (5×5). Figures 1 and 2 show the performance in a smaller grid environment with fewer obstacles. The reward function is such that the agent receives a reward of 1 if it reaches the goal state, and otherwise receives a reward of -0.1. Bumping into obstacles only results in the agent's position remaining unchanged. In this simple setting, the difference between our method and retraining from scratch is not very large. Both approaches converge quickly to similar performance because the state–action space is limited, making adaptation relatively easy. Nevertheless, our method still maintains slightly higher initial returns and fewer bumps, confirming that it does not hurt performance even in simple environments.

Large GridWorld (25×25). Figures 3, 4, 5 and 6 show the results in a more complex environment with many obstacles in both tabular and DQN approaches. Here, the advantages of our method are much clearer. Compared to retraining and naive expansion, our method adapts faster, achieves higher discounted returns, and results in fewer collisions. The larger state– action space magnifies the cost of retraining, while knowledge reuse provides a strong benefit. This confirms that our algorithm is especially effective in complex and dynamic environments.

Overall, these experiments suggest that while the gains in small environments are modest, the benefits of our approach become significant as the environment grows larger and more challenging. This highlights the promise of knowledge reuse for real-world applications where action sets expand in complex domains.

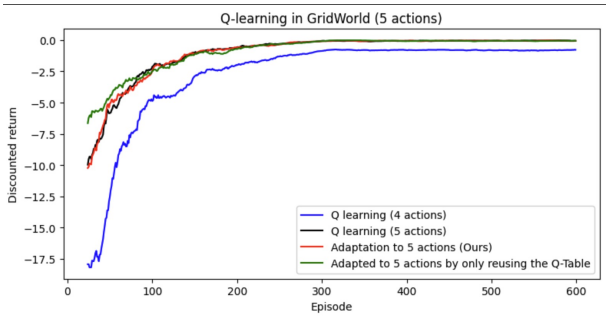

Fig. 1. Discounted return in GridWorld (5×5). The difference between our method and retraining from scratch is small, as the environment is simple.

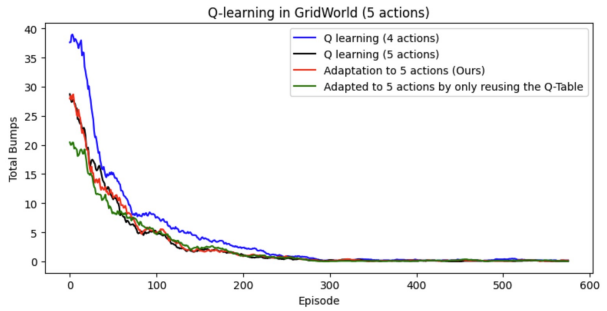

Fig. 2. Total bumps in GridWorld (5×5). Our method slightly reduces collisions, but the improvement over retraining is modest.

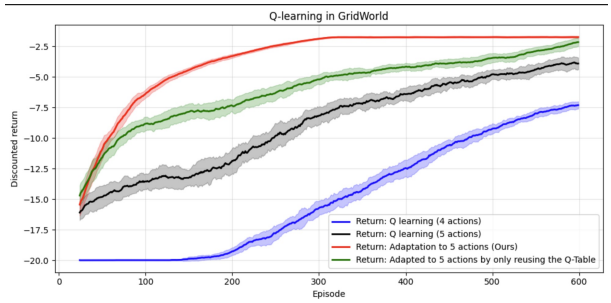

Fig. 3. Discounted return in GridWorld (25×25) (Tabular Setting). Our method works better in larger and more complex environments.

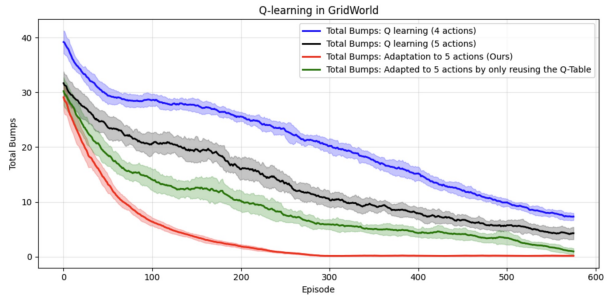

Fig. 4. Discounted return in GridWorld (25×25) (Tabular Setting). Our method achieves much safer adaptation compared to baselines.

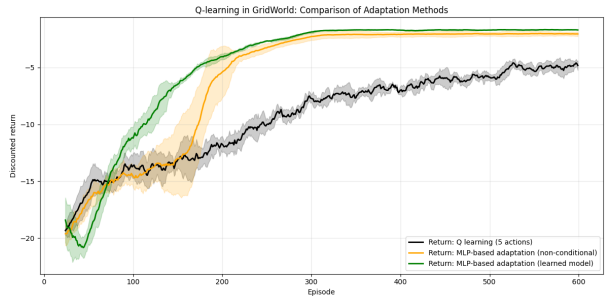

Fig. 5. Total bumps in GridWorld (25×25) (DQN Setting). Our method achieves much better adaptation compared to baselines.

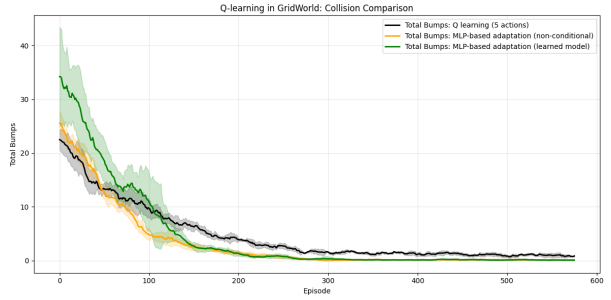

Fig. 6. Total bumps in GridWorld (25×25) (DQN Setting). Our method achieves much safer adaptation compared to baselines.

TABLE I
(TABULAR-ORACLE SETTING: COMPARISON OF PERFORMANCE ACROSS DIFFERENT METHODS. REPORTED VALUES ARE AVERAGED OVER 5 RUNS. HIGHER DISCOUNTED RETURN AND LOWER BUMPS INDICATE BETTER PERFORMANCE.

Method	Discounted Return	Total Bumps
Q-learning (4 actions)	-7.7 ± 0.3	7.3 ± 0.6
Q-learning (5 actions)	-4.0 ± 0.5	4.3 ± 1.1
Adaptation to 5 actions (Ours)	-1.7 ± 0.0	0.2 ± 0.1
Adapted to 5 actions by only reusing the Q-Table	-2.6 ± 0.4	1.0 ± 0.6

TABLE II
(DQN-MLP SETTING: COMPARISON OF PERFORMANCE ACROSS DIFFERENT METHODS. REPORTED VALUES ARE AVERAGED OVER 5 RUNS. HIGHER DISCOUNTED RETURN AND LOWER TOTAL BUMPS INDICATE BETTER PERFORMANCE.

Method	**Discounted Return**	**Total Bumps**
Q-learning (4 actions)	-20.06 ± 0.03	0.60 ± 0.27
Q-learning (5 actions, retrained)	-5.11 ± 0.48	1.00 ± 0.30
Adaptation to 5 actions (Ours)	$\mathbf{-1.68 \pm 0.03}$	$\mathbf{0.11 \pm 0.07}$
Adaptation to 5 actions (No filtering)	-2.03 ± 0.20	0.16 ± 0.08

VI. DISCUSSION AND CONCLUSION

This paper addressed the challenge of adapting reinforcement learning agents to expanding action spaces without retraining from scratch. We proposed a Q-learning based framework that reuses previously learned policies and value functions when new actions are introduced. Three extensions were developed: optimistic initialization for new actions, a partial model that predicts the next state for newly introduced actions and a selection mechanism that integrates new actions only when they improve the estimated state value. Experimental evaluation in a GridWorld environment demonstrated that our method outperforms baselines in both efficiency and safety, achieving higher discounted returns and fewer bumps compared to retraining or naive expansion of Q-learning. These results confirm that leveraging prior knowledge provides significant benefits when adapting to dynamic action spaces.

Overall, this work contributes toward continual reinforcement learning by showing that action-space adaptation can be achieved through knowledge reuse. By combining efficiency, and safety, our method provides a practical step toward agents that can operate robustly in dynamic and evolving environments.

REFERENCES

[1] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An introduction*. MIT press Cambridge, 1998, vol. 1, no. 1.

[2] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.

[3] Y. Chandak, G. Theocharous, C. Nota, and P. Thomas, "Lifelong learning with a changing action set," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 3373–3380.

[4] A. Jain, A. Szot, and J. J. Lim, "Generalization to new actions in reinforcement learning," *arXiv preprint arXiv:2011.01928*, 2020.

[5] T. Mandel, Y.-E. Liu, E. Brunskill, and Z. Popović, "Where to add actions in human-in-the-loop reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 31, no. 1, 2017.

[6] H. P. Van Hasselt, M. Hessel, and J. Aslanides, "When to use parametric models in reinforcement learning?" *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[7] R. Zhang, H. Fu, Y. Miao, and G. Konidaris, "Model-based reinforcement learning for parameterized action spaces," *arXiv preprint arXiv:2404.03037*, 2024.

[8] W. Ding, S. Jiang, H.-W. Chen, and M.-S. Chen, "Incremental reinforcement learning with dual-adaptive ε-greedy exploration," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, 2023, pp. 7387–7395.

[9] C. Pan, X. Yang, Y. Li, W. Wei, T. Li, B. An, and J. Liang, "A survey of continual reinforcement learning," *arXiv preprint arXiv:2506.21872*, 2025.

[10] W. H. Delashmit, M. T. Manry *et al.*, "Recent developments in multilayer perceptron neural networks," in *Proceedings of the seventh annual memphis area engineering and science conference, MAESC*, vol. 7, 2005, p. 33.